Claims Management Application

QA findings: bugs found, live evidence and fixes

Prepared by	Chan Basha
For	Chubb APAC QA Engineer take-home, hiring panel walkthrough
Date	12 October 2026
Repository	github.com/chansgrowthhub-arch/chubb-claimns-qa-assessment
Baseline	Commit `1dea234`: the application exactly as received
Environment	Windows 11, Docker Desktop 4.94, JDK 21, Maven 3.9.9, Node 22

> **How to read this report.** Every screenshot is a real artefact. Terminal images are captured from actual runs: unit, integration, E2E and load suites, plus live HTTP and WebSocket probes against the running stack. Code images are read from the source at the line numbers shown, and diffs come from `git diff`. "Before" means the untouched baseline commit.

> **Scope.** The full Docker Compose stack was started and tested live, including the optional Debezium CDC stack. Every high-severity finding was reproduced on the running system. Two of the recommended fixes were validated live by restarting a single container with the corrected setting.

1. Summary

18
findings, 5 of them high severity

4
fixed in this work

2
recommended fixes validated live

11/11
E2E specs pass on the live stack (2 known-defect tests fail as expected)

#	Finding	Severity	Status	Proof
1	Real-time updates are off in the default stack	High	Open	Live; fix validated live
2	Signup is broken: wrong Keycloak admin password	High	Open	Live; fix validated live
3	A fresh clone renders no app pages: API client never generated	High	Open	Live, from a fresh clone
4	Logout rejected by CSRF protection; session survives	High	Open	Live API and browser; E2E
5	Logout fails when the access token has expired	High	Open	Live; failing defect test
6	CDC contract wrong on 5 fields (schema says `id`, Debezium sends `claim_id`)	Medium	Open	Real Debezium message
7	BFF returns 500 for "not found" and "forbidden" claims	Medium	Open	Live; E2E
8	Incident dates one day early west of UTC	Medium	Open	Reproduced; unit test
9	The shipped test suite could not pass	Medium	Fixed	Before and after runs
10	Shared bearer token across concurrent requests	Medium	Open	Code review
11	`changedBy` records the claim owner instead of the admin	Medium	Open	Code review
12	Theme preference lost in every new tab	Low	Fixed	Shipped tests pass
13	E2E runner broken: setup file missing	Low	Fixed	11 specs run
14	Unpinned `:latest` Testcontainers images break integration tests	Low	Fixed	Before and after runs
15	Concurrent logins for the same account rejected	Low	Open	Live
16	`start-cdc.sh` reports failure on a clean, successful run	Low	Open	Live
17	Status-change error detail discarded by the BFF	Low	Open	Code review
18	Smaller contract and security smells	Low	Open	Code review

Fixed means committed and verified by a passing run. Three of the fixes were to test infrastructure and one was a one-line application change. **Open** means the bug is still in the application. As QA, I proved and pinned these bugs rather than silently changing product behaviour. Each open bug has a recommended fix in this report, and five of them also have a failing test that turns green once the fix lands.

2. Test results on the live stack

Suite	As received	After this work
Claims service unit	Did not compile; then 9 failures	237 passed
BFF unit	213 passed, but blind to findings 4–6	219 passed, plus 2 defect tests failing by design
UI component and unit	8 failed, 258 passed	280 passed, including 1 known-defect test
Integration (Testcontainers)	2 errors on this machine	Claims service 8 passed, BFF 4 passed
Playwright E2E	Runner broken	11 passed: 9 green and 2 known-defect tests failing as expected
k6 load	Not present	650 requests, 0 failed; list-claims p95 72 ms

All three suites - current HEAD

```
$ mvn test  (claims-service)  |  mvn test  (bff-service)  |  npm test  (demo-app-ui)
# claims-service
[INFO] Tests run: 237, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

# bff-service
[INFO] Tests run: 219, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS

# demo-app-ui
Test Files  28 passed (28)
Tests  280 passed (280)
```

Figure 2.1: Default unit and component suites, current code.

Playwright E2E on the live Docker Compose stack (UI container restarted cold first)

```
$ npx playwright test
  ok  4 auth-and-rbac.spec.ts:19:7 › authentication › redirects anonymous visitors from protected routes to login (2.7s)
  ok  7 auth-and-rbac.spec.ts:50:7 › role-based access › claimant never receives claim data for an id they do not own (2.5s)
  ok  8 auth-and-rbac.spec.ts:44:7 › role-based access › claimant calling an admin API directly receives 403 (2.6s)
  x   2 auth-and-rbac.spec.ts:64:7 › role-based access › missing claim returns 404 as the BFF contract specifies (3.0s)
  ok  3 auth-and-rbac.spec.ts:6:7 › authentication › rejects wrong credentials with a clear message (3.7s)
  ok  5 auth-and-rbac.spec.ts:14:7 › authentication › routes each role to its landing page (3.7s)
  ok  6 claims-lifecycle.spec.ts:30:5 › claimant submits a claim and sees it listed as Submitted (3.5s)
  x   1 auth-and-rbac.spec.ts:28:7 › authentication › logout invalidates the server session (4.1s)
  ok  9 claims-lifecycle.spec.ts:48:5 › admin moves the claim to Under Review and then Approved (1.3s)
  ok 10 claims-lifecycle.spec.ts:61:5 › admin is not offered illegal transitions for an Approved claim (783ms)
  ok 11 claims-lifecycle.spec.ts:71:5 › claimant sees the approved status after reload (783ms)
  11 passed (17.6s)
```

Figure 2.2: Playwright on the live stack. The UI container was restarted cold first. The "x" rows are the two known-defect tests (findings 4 and 7), which Playwright counts as expected failures.

Testcontainers integration tests on Docker Desktop

```
$ mvn test -Dgroups=integration -Dsurefire.excludedGroups=
# BEFORE - claims-service with images as shipped (postgres:latest = PG 18.6)
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 -- adapter.out.persistence.ClaimRepositoryIntegrationTest
[ERROR] Tests run: 1, Failures: 0, Errors: 1, Skipped: 0 <<< FAILURE! -- ClaimsServiceApplicationTests
[ERROR] Tests run: 1, Failures: 0, Errors: 1, Skipped: 0 <<< FAILURE! -- HealthCheckIntegrationTest
Caused by: org.postgresql.util.PSQLException: FATAL: invalid value for parameter "TimeZone": "Asia/Calcutta"

# AFTER pinning postgres:15-alpine and redis:7-alpine - claims-service, then bff-service
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 -- adapter.out.persistence.ClaimRepositoryIntegrationTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- ClaimsServiceApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- HealthCheckIntegrationTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- BffServiceApplicationTests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0 -- adapter.in.messaging.ClaimEventKafkaConsumerIntegrationTest
```

Figure 2.3: Integration tests before and after pinning the images (finding 14).

k6 on the live stack - 10 claimant VUs + 1 admin VU for 60 s

```
$ docker run --network demo-network grafana/k6:0.54.0 run -e VUS=10 -e DURATION=60s /scripts/k6-claims-smoke.js
     ✓ login succeeded
       ✓ list claims 200
       ✓ list claims returns array
       ✓ dashboard 200
   ✓ checks.........................: 100.00% 1232 out of 1232
     ✓ { endpoint:dashboard_cached }...: avg=46.5ms  min=21.49ms med=42.05ms  max=102.05ms p(90)=66.87ms  p(95)=76.97ms
     ✓ { endpoint:list_claims }.......: avg=26.44ms min=5.69ms  med=19.37ms  max=141.94ms p(90)=52.44ms  p(95)=72.09ms
   ✓ http_req_failed................: 0.00%   0 out of 650
     http_reqs.....................: 650     10.664289/s
     iterations....................: 639     10.483817/s
```

Figure 2.4: k6 on the live stack. One Keycloak account per virtual user (finding 15). Thresholds are first-pass laptop values, not an SLA.

> **Load-test honesty.** The first k6 runs failed for reasons in my own script, not the app. k6 clears cookies every iteration by default, and a failing check skipped the think time. I fixed both before trusting any number. The runs also exposed finding 15.

3. Finding 1: Real-time updates are off in the default stack

High **Open**

What happens A claim is submitted or a status changes, and no live update reaches anyone. No toast appears and the list does not refresh until a reload. Nothing errors.

Why The compose file turns CDC mode on, so the claims service publishes no Kafka events and relies on Debezium. Debezium is an optional stack that the quick-start never starts. The README says the opposite default.

docker-compose.apps.yml

```
16      environment:
17        SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/demo_app
18        SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:29092
19        SPRING_DATA_REDIS_HOST: redis
20        SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_ISSUER_URI: http://localhost:8180/realms/demo-app
21        SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_JWK_SET_URI: http://keycloak:8080/realms/demo-app/protocol/openid-connect/certs
22        # Set to true when running docker-compose.cdc.yml alongside this stack
23        APP_EVENTS_CDC_ENABLED: "true"
24      networks:
```

Compose sets **CDC mode ON** for claims-service ...

Figure 3.1: CDC is on in the shipped compose file.

code/claims-service/src/main/java/com/example/demo/application/usecases/CreateClaimUseCase.java

```
47          Claim savedClaim = claimRepository.save(claim);
48
49          if (cdcEnabled) {
50              LOG.info("CDC enabled — event will be published by Debezium for claimId={}", savedClaim.getClaimId());
51          } else {
52              LOG.info("CDC disabled — publishing {} event(s) directly for claimId={}",
53                      claim.getDomainEvents().size(), savedClaim.getClaimId());
54              claim.getDomainEvents().forEach(eventPublisher::publishEvent);
55              claim.clearEvents();
56          }
```

With cdcEnabled=true the service only logs - it publishes nothing and relies on Debezium, which the quick-start never starts.

Figure 3.2: With CDC on, the use case only writes a log line.

Live stack as shipped - Bug 1 (real-time updates)

```
$ node verify-live.cjs realtime   +   docker logs   +   kafka-get-offsets
08:20:04 === Bug 1: does the admin receive CLAIM_SUBMITTED in real time? ===
08:20:08 login admin: HTTP 200, role=ADMIN
08:20:08 login claimant: HTTP 200, role=CLAIMANT
08:20:08 websocket admin: connected
08:20:09 submit claim as claimant: HTTP 201, claimId=ed1cec5d-8487-4b50-a576-4e98a5b2a439
08:20:24 RESULT: no WebSocket event for ed1cec5d-8487-4b50-a576-4e98a5b2a439 within 15s -> real-time pipeline BROKEN
# claims-service log for the submitted claim
2026-10-12 08:20:09.187 [http-nio-8080-exec-10] INFO  c.e.d.a.usecases.CreateClaimUseCase [correlationId=fe8b8f0f-8a9b-4452-923a-
7c9be94f1920] - CDC enabled — event will be published by Debezium for claimId=ed1cec5d-8487-4b50-a576-4e98a5b2a439
# Kafka topics on the broker
cdc.demo-app.public.claims
claim-events
# end offsets of claim-events (0 or missing = nothing ever published)
claim-events:0:0
# Debezium container running?
0
```

Figure 3.3: Live proof. The admin WebSocket gets nothing, the service logs "will be published by Debezium", `claim-events` holds 0 messages, and no Debezium container is running.

Recommended fix, validated live

```
docker-compose.apps.yml  (claims-service)
-       APP_EVENTS_CDC_ENABLED: "true"
+       APP_EVENTS_CDC_ENABLED: "false"   # direct Kafka by default; set true only with ./start-cdc.sh
```

Same stack, claims-service restarted with APP_EVENTS_CDC_ENABLED=false (recommended fix)

```
$ node verify-live.cjs realtime   +   kafka-get-offsets
08:21:26 === Bug 1: does the admin receive CLAIM_SUBMITTED in real time? ===
08:21:27 login admin: HTTP 200, role=ADMIN
08:21:27 login claimant: HTTP 200, role=CLAIMANT
08:21:27 websocket admin: connected
08:21:28 submit claim as claimant: HTTP 201, claimId=4f49b970-374f-444c-92de-1047c5a5bf45
08:21:28 websocket admin: received {"type":"CLAIM_SUBMITTED","claimId":"4f49b970-374f-444c-92de-
1047c5a5bf45","newStatus":"SUBMITTED","oldStatus":null,"timestamp":"2026-10-12T08:21:28.452101162Z","correlationId":"7effcf5c-95aa-4dfb-
96a6-626f3e90c5fd"}
08:21:28 RESULT: admin received CLAIM_SUBMITTED for 4f49b970-374f-444c-92de-1047c5a5bf45 -> real-time pipeline WORKS
claim-events:0:1
```

Figure 3.4: The same check after restarting only the claims service with the fix. The admin receives `CLAIM_SUBMITTED` within a second, and the topic now holds 1 message.

Test coverage. `CreateClaimUseCaseTest` asserts that the event is published and then cleared when CDC is off. The live probe above is the end-to-end check.

CDC mode itself also works once Debezium is started; see Figure 8.4.

4. Finding 2: Signup is broken

High **Open**

What happens	Every new-user signup returns HTTP 500. No one can register.
Why	Signup creates the user through the Keycloak admin API. The BFF defaults the admin password to `admin`, but Keycloak is bootstrapped with `Admin123!`, and the apps compose file never passes the real value.
How found	Only on the live stack. Unit tests mock the Keycloak adapter, so they cannot see this.



Figure 4.1: BFF default.

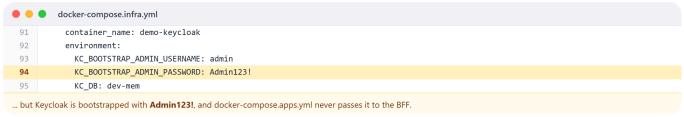


Figure 4.2: Keycloak bootstrap.

```
Live stack as shipped - signup

$ curl POST /api/auth/signup   +   docker logs demo-bff-service   +   Keycloak token checks
# POST /api/auth/signup on the stack as shipped
signup new user: HTTP 500
# BFF log
threw exception [Request processing failed: jakarta.ws.rs.ProcessingException: jakarta.ws.rs.NotAuthorizedException: HTTP 401 Unauthorized
at com.example.demo.adapter.out.keycloak.KeycloakAdminAdapter.createUser(KeycloakAdminAdapter.java:52)
# Keycloak master-realm admin token, both candidate passwords
password 'admin': HTTP 401
password 'Admin123!': HTTP 200
# what each side is configured with
application.yml:69:    password: ${KEYCLOAK_ADMIN_PASSWORD:admin}
docker-compose.infra.yml:94:      KC_BOOTSTRAP_ADMIN_PASSWORD: Admin123!
docker-compose.apps.yml bff-service KEYCLOAK_ADMIN_PASSWORD entries: 0
```

Figure 4.3: Signup returns 500, the BFF log shows the Keycloak admin client rejected with 401, and Keycloak accepts only `Admin123!`.

Recommended fix, validated live

```
docker-compose.apps.yml  (bff-service.environment)
+      KEYCLOAK_ADMIN_PASSWORD: "Admin123!"   # better: one shared .env value for both services
```

After restarting only the BFF with this setting, signup returns 201 and the new user can log in and create a claim. See the first lines of Figure 9.2.

5. Finding 3: A fresh clone renders no app pages

High **Open**

What happens	Following the README on a fresh clone, the login page loads, but every page behind login returns HTTP 500.
Why	The UI imports a client generated from the OpenAPI spec. That folder is gitignored, and neither the Dockerfile nor `start-app.sh` generates it.
How found	It worked on my machine only because I had generated the client earlier. I proved it by cloning the repo from GitHub and building the UI image exactly as compose does.

code/demo-app-ui/Dockerfile

```
1  FROM node:20-alpine
2  WORKDIR /app
3  COPY package*.json ./
4  RUN npm ci
5  COPY . .
6  EXPOSE 3001
7  CMD ["npx", "next", "dev", "-H", "0.0.0.0", "-p", "3001"]
```

The image copies the folder and runs **next dev**. Nothing runs npm run generate:api, and the generated client is gitignored.

Figure 5.1: The UI image never generates the client.

Fresh clone from GitHub, UI image built exactly as docker compose does

```
$ git clone ... && docker build code/demo-app-ui && curl /login /claims
# generated API client present in the fresh clone: 0 files (src/lib/api/generated is gitignored)
GET /login  -> 200
GET /claims -> 500
# container log:
Module not found: Can't resolve './generated'
Module not found: Can't resolve '@/src/lib/api/generated/runtime'
```

Figure 5.2: Fresh clone: `/login` returns 200, `/claims` returns 500, and the container logs "Module not found".

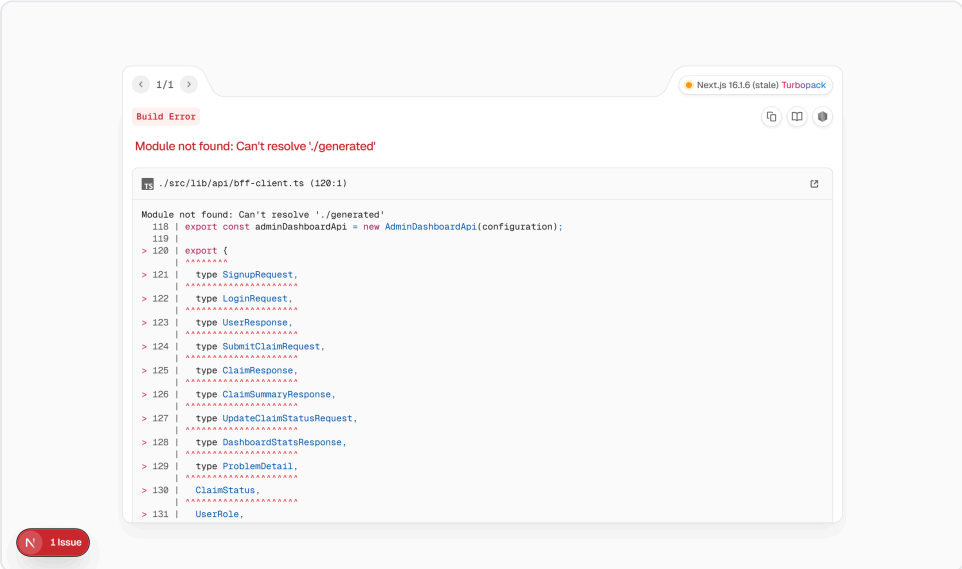


Figure 5.3: What a user sees on `/claims` from a fresh clone.

Recommended fix

```
start-app.sh, before "docker compose ... up --build":
+  (cd "$SCRIPT_DIR/code/demo-app-ui" && npm ci && npm run generate:api)
Better: generate inside the image, with the build context set to code/ so the spec is available.
Add a CI job that builds from a fresh clone so this class of bug cannot return.
```

6. Finding 4: Logout is rejected by CSRF protection

High **Open**

What happens	The user clicks Logout and lands on the login page, so they look logged out. Their session cookie survives, and opening `/claims` again still shows their claims.
Why	Logout is not on the BFF's CSRF exemption list, and the UI sends a plain fetch without the `X-XSRF-TOKEN` header. The BFF answers 403, and the UI clears its local state anyway.
Why it slipped	The shipped controller test always attached a CSRF token, so it never sent the request the browser actually sends.

code/bff-service/src/main/java/com/example/demo/config/SecurityConfig.java

```
55                  .csrf(csrf -> csrf
56                          .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
57                          .csrfTokenRequestHandler(csrfTokenRequestHandler)
58                          .ignoringRequestMatchers(
59                                  "/api/auth/signup",
60                                  "/api/auth/login",
61                                  "/api/auth/refresh",
62                                  "/api/ws/**",
63                                  "/actuator/**"
64                          ))
```

CSRF exemptions: signup, login, refresh ... **logout is not listed**, so it requires an X-XSRF-TOKEN header.

Figure 6.1: Logout is missing from the CSRF exemptions.

code/demo-app-ui/src/components/layout/app-header.tsx

```
16  const handleLogout = async () => {
17    setIsLoggingOut(true);
18    try {
19      await fetch('/api/auth/logout', {
20        method: 'POST',
21        credentials: 'include',
22      });
23    } catch (error) {
24      console.error('Logout error:', error);
25    }
26
27    logout();
28    router.push('/login');
29  };
```

UI logout: raw fetch with **no CSRF header** -> BFF answers 403 -> the catch never fires (403 is not an exception) -> local state is cleared anyway.

Figure 6.2: The UI logout sends no CSRF header and ignores the 403.

Live stack as shipped - Bugs 3 and 4 (logout)

```
$ node verify-live.cjs logout   +   real Chromium login, then click "Logout"
08:20:24 === Bug 3: logout exactly as the UI sends it (cookie, no CSRF header) ===
08:20:24 login claimant: HTTP 200, role=CLAIMANT
08:20:24 GET /api/claims before logout: HTTP 200
08:20:24 POST /api/auth/logout (no X-XSRF-TOKEN): HTTP 403, Set-Cookie headers: 0
08:20:24 GET /api/claims after "logout" with same cookie: HTTP 200  -> session SURVIVED logout
08:20:24 === Bug 4: logout with an expired/invalid access token ===
08:20:24 POST /api/auth/logout with stale token + valid CSRF: HTTP 401, Set-Cookie headers: 0  -> cookies NOT cleared
# real browser: fresh login, click Logout, then open /claims again
browser logout: POST /api/auth/logout -> HTTP 403; access_token cookie still present: true; navigating back to /claims lands on /claims; GET
/api/claims -> HTTP 200
```

Figure 6.3: Live proof through the API and a real browser. Logout returns 403, the session cookie survives, and `/claims` still answers 200. The same capture shows finding 5: a stale token gets 401 and no cookies are cleared.

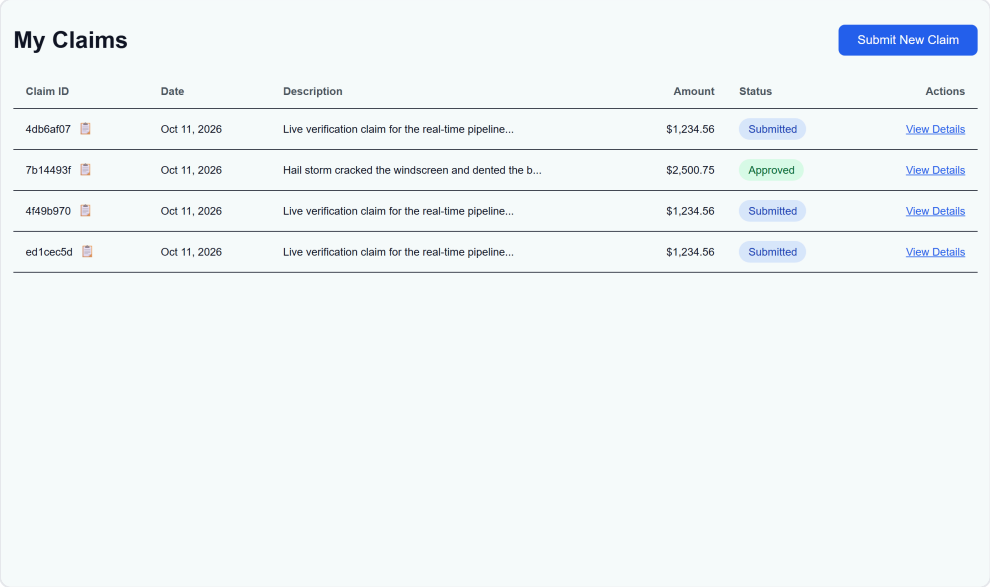


Figure 6.4: Real browser, straight after clicking Logout. The header has gone because the app thinks the user is logged out, but their claims still load.

Recommended fix

```
app-header.tsx: send the CSRF header, as the generated API client already does
  const csrf = document.cookie.match(/XSRF-TOKEN=([^;]+)/)?.[1];
  const res = await fetch('/api/auth/logout', { method: 'POST', credentials: 'include',
    headers: csrf ? { 'X-XSRF-TOKEN': decodeURIComponent(csrf) } : {} });
  if (!res.ok) { show an error; do not pretend the user is logged out }
```

Test coverage. `AuthCsrfContractTest` pins the server contract. The Playwright test `logout invalidates the server session` is marked as an expected failure until this is fixed.

7. Finding 5: Logout fails when the access token has expired

High Open

What happens	A user whose 5-minute access cookie is stale cannot log out. The BFF answers 401 before the logout code runs, so the 30-minute refresh cookie is never cleared.
Why	A filter turns every `access_token` cookie into a Bearer header, and Spring rejects the expired token before the controller is reached.
How found	Writing the passing CSRF test, my first version included a cookie and got 401. I kept that case as a defect test rather than deleting the cookie to make the test pass.

code/bff-service/src/main/java/com/example/demo/config/CookieAuthenticationFilter.java

```
36          // Extract access_token from cookies
37          String accessToken = extractAccessTokenFromCookies(request);
38
39          if (accessToken != null) {
40              // Wrap request to add Authorization header
41              HttpServletRequest wrappedRequest = new HttpServletRequestWrapper(request) {
42                  @Override
43                  public String getHeader(String name) {
44                      if (AUTHORIZATION_HEADER.equalsIgnoreCase(name)) {
45                          return BEARER_PREFIX + accessToken;
46                      }
47                      return super.getHeader(name);
48                  }
49              };
50              filterChain.doFilter(wrappedRequest, response);
```

Any access_token cookie becomes a Bearer header. An **expired** token is rejected with 401 before logout ever runs.

Figure 7.1: Every access_token cookie becomes an Authorization header.

bff-service - known-defect tests (tag "defect"), FAIL BY DESIGN

```
$ mvn test -Dgroups=defect -Dsurefire.excludedGroups=integration
[ERROR]   ClaimEventKafkaConsumerContentTest.should_BroadcastClaimId_When_CdcPayloadMatchesCommittedAsyncApiSchema:148 [claimId must survive
a schema-conformant CDC payload (currently arrives empty)]
expected: "d4e5f6a7-b8c9-0123-4567-89abcdef0123"
 but was: ""
[ERROR]   AuthCsrfContractTest.should_ClearCookies_When_LogoutCalledWithExpiredAccessToken:96 Status expected:<204> but was:<401>
[ERROR] Tests run: 2, Failures: 2, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
```

Figure 7.2a: The two BFF defect tests fail by design. The second is this finding; the first is finding 6.

Recommended fix

```
SecurityConfig: .requestMatchers("/api/auth/logout").permitAll()
CookieAuthenticationFilter: skip the cookie-to-bearer conversion for /api/auth/logout
AuthController.logout: always clear both cookies; invalidate the Keycloak session best-effort
```

Live confirmation is in Figure 6.3: a stale token plus a valid CSRF token gets 401 and zero `Set-Cookie` headers.

8. Finding 6: The CDC message contract is wrong on 5 fields

Medium **Open**

What happens	Change events built from the committed contract reach the browser with an empty claim id. That includes every event from the test-only injection endpoint. Any consumer that trusts the schema decodes the amount and dates wrongly.
Why	The BFF reads `claim_id`, the real Debezium column name. The committed AsyncAPI schema and the injection endpoint use `id`, and the schema's types for amount and dates do not match what Debezium emits.
Severity change	I rated this High from code review. On the live stack, real CDC works end to end, so the production path is fine. I moved it to Medium and widened its scope.

code/bff-service/src/main/java/com/example/demo/adapter/in/messaging/ClaimEventKafkaConsumer.java

```
114     private void handleCdcUpdate(JsonNode cdcEnvelope) {
115         JsonNode before = cdcEnvelope.path("before");
116         JsonNode after = cdcEnvelope.path("after");
117
118         String claimId = after.path("claim_id").asText();
119         String newStatus = after.path("status").asText();
120         String oldStatus = before.path("status").asText();
121         String userId = after.path("user_id").asText(null);
```

BFF consumer reads the claim id from **after.claim_id**.

Figure 8.1: The consumer expects `claim_id`.

code/claims-service/src/main/resources/asyncapi/schemas/DebeziumClaimsMessage.json

```
74            "type": "object",
75            "description": "Shared claim fields reused in CDC after schema and domain events",
76            "properties": {
77              "id": {
78                "type": "string",
79                "format": "uuid"
80              },
81              "user_id": {
```

The committed message contract names the same field **id**.

Figure 8.2: The schema documents `id`.

code/claims-service/src/main/java/com/example/demo/adapter/in/web/TestEventController.java

```
62          return """
63              {
64                "op": "u",
65                "after": {
66                  "id": "%s",
67                  "status": "%s",
68                  "updated_at": %d
69                },
```

The E2E event-injection endpoint also emits **id** - so every injected event reaches the UI with an empty claimId.

Figure 8.3: The E2E injection endpoint emits `id`.

Live stack with Debezium CDC running - a real change event

```
$ node verify-live.cjs realtime   +   kafka-console-consumer cdc.demo-app.public.claims
08:33:49 === Bug 1: does the admin receive CLAIM_SUBMITTED in real time? ===
08:33:49 login admin: HTTP 200, role=ADMIN
08:33:49 login claimant: HTTP 200, role=CLAIMANT
08:33:50 websocket admin: connected
08:33:50 submit claim as claimant: HTTP 201, claimId=4db6af07-374f-4a03-b672-345fd2a58799
08:33:50 websocket admin: received {"type":"CLAIM_SUBMITTED","claimId":"4db6af07-374f-4a03-b672-
345fd2a58799","newStatus":"SUBMITTED","oldStatus":null,"timestamp":"2026-10-12T08:33:50.284Z","correlationId":"dfe11412-e56a-44c5-aef2-
3cf942bd192a"}
08:33:50 RESULT: admin received CLAIM_SUBMITTED for 4db6af07-374f-4a03-b672-345fd2a58799 -> real-time pipeline WORKS
op = c | top-level keys = ['after', 'before', 'op', 'source', 'transaction', 'ts_ms']
after keys = ['claim_amount', 'claim_id', 'created_at', 'description', 'incident_date', 'incident_location', 'status', 'updated_at',
'user_id']
after.claim_id = 4db6af07-374f-4a03-b672-345fd2a58799 | after.id = None
# field: real Debezium value (python type)  vs  committed DebeziumClaimsMessage.json
claim_amount  : 'AeJA'                              (str  )  vs schema: number
incident_date : 20737                               (int  )  vs schema: string, format date
created_at    : 1791794030102982                    (int  )  vs schema: string, format date-time
updated_at    : 1791794030102982                    (int  )  vs schema: string, format date-time
status        : 'SUBMITTED'                         (str  )  vs schema: enum string
```

Figure 8.4: A real Debezium event from the live stack. The admin receives it in real time, the payload carries `claim_id` with no `id`, the amount arrives as base64 decimal bytes (`"AeJA"` is 1234.56), and the dates arrive as epoch days or microseconds.

Recommended fix

```
DebeziumClaimsMessage.json: id -> claim_id; claim_amount -> base64 decimal bytes (or set
  decimal.handling.mode=string in the connector); dates -> integer epoch days / micros
TestEventController: emit "claim_id" and the real types
CI: validate a captured real Debezium message against the schema (the validator already exists)
```

Test coverage. `ClaimEventKafkaConsumerContentTest` asserts message content. Its `defect`-tagged case fails against the documented shape (Figure 7.2a).

9. Finding 7: BFF returns 500 for "not found" and "forbidden" claims

Medium **Open**

The claims service correctly answers 404 or 403 with a ProblemDetail. The BFF rethrows both as a generic exception, so the browser gets a bare 500 and the UI shows "Failed to load" instead of "Claim not found". No data leaks; the guard test confirms that.

code/bff-service/src/main/java/com/example/demo/adapter/out/claimsservice/ClaimsServiceAdapter.java

```
170     public ClaimRecord getClaim(String userJwtToken, UUID claimId) {
171         setUserBearerToken(userJwtToken);
172
173         try {
174             var response = claimsApi.getClaim(claimId);
175             return mapToClaimRecord(response);
176         } catch (ApiException exception) {
177             if (exception.getCode() == 403) {
178                 throw new IllegalStateException("Forbidden: not authorized to access this claim", exception);
179             }
180             if (exception.getCode() == 404) {
181                 throw new IllegalStateException("Claim not found: " + claimId, exception);
182             }
183             throw new IllegalStateException("Claims Service call failed: " + exception.getMessage(), exception);
184         }
185     }
```

claims-service 403/404 ProblemDetails are rethrown as **IllegalStateException**, which Spring turns into a bare 500.

Figure 9.1: Both error codes are converted to `IllegalStateException`.

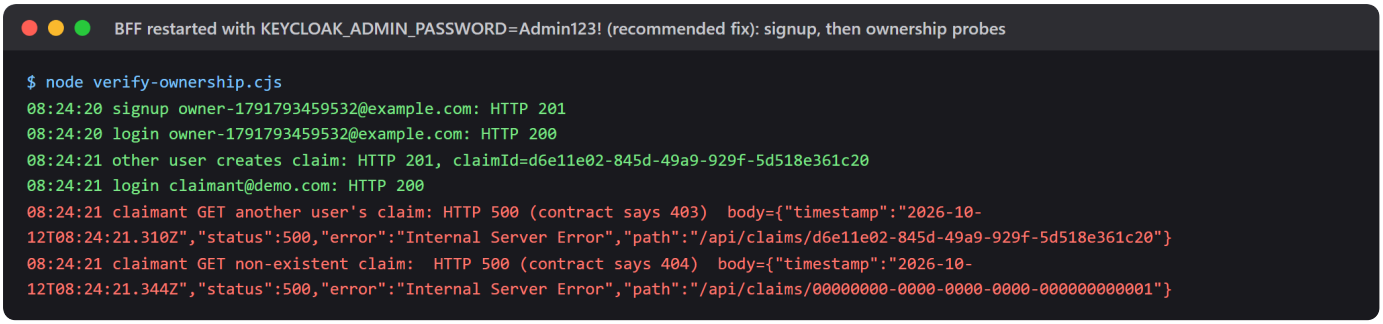


Figure 9.2: Live probe, run after the finding 2 fix so a second user could sign up. Another user's claim returns 500 where the contract says 403, and a missing claim returns 500 where it says 404.

Fix. Map claims-service ProblemDetails through the BFF with their original status, via a typed exception plus a `@RestControllerAdvice`. **Test.** The Playwright test `missing claim returns 404 as the BFF contract specifies` is marked as an expected failure.

10. Finding 8: Incident dates show one day early west of UTC

Medium **Open**

Incident date is a date-only value. The generated client parses `"2026-10-01"` as UTC midnight, and the UI formats it in local time, so users in the Americas see the previous day. APAC and UK users are unaffected, which is why it is easy to miss from Asia.

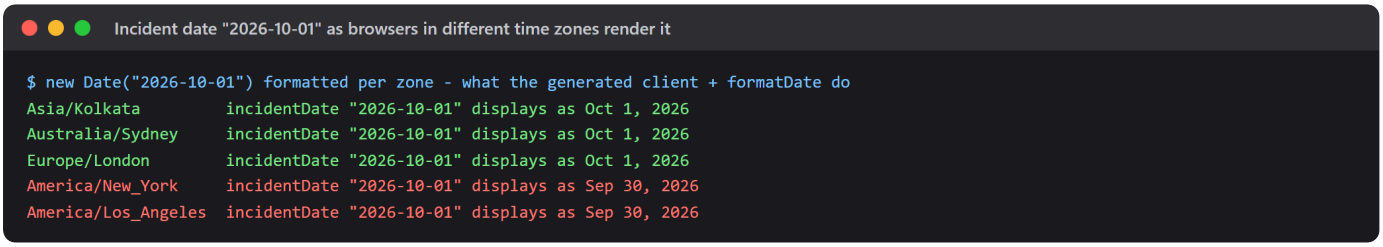


Figure 10.1: The same incident date in five time zones.

Fix. Treat date-only fields as strings, or format with `timeZone: 'UTC'`. **Test.** `incident-date-timezone.test.ts` uses `it.fails`, so CI stays green until the fix lands and then turns red.

11. Finding 9: The shipped test suite could not pass

Medium **Fixed**

The brief mentioned one planted broken test, but the problem was wider. No CI pipeline could have been green, so the configured 90% coverage and 80% mutation thresholds were never enforced.

claims-service - as received (baseline commit 1dea234)

```
$ mvn test
[ERROR] COMPILATION ERROR :
[INFO] -------------------------------------------------------------
[ERROR] code/claims-service/src/test/java/com/example/demo/application/usecases/UpdateClaimStatusUseCaseTest.java:[45,19] constructor
UpdateClaimStatusUseCase in class com.example.demo.application.usecases.UpdateClaimStatusUseCase cannot be applied to given types;
  required:
com.example.demo.domain.claim.ClaimRepository,com.example.demo.user.domain.UserRepository,io.micrometer.core.instrument.MeterRegistry,org.sp
ringframework.context.ApplicationEventPublisher
  found:
com.example.demo.domain.claim.ClaimRepository,com.example.demo.user.domain.UserRepository,io.micrometer.core.instrument.MeterRegistry
  reason: actual and formal argument lists differ in length
[INFO] -------------------------------------------------------------
[INFO] BUILD FAILURE
```

Figure 11.1: The baseline does not compile. The test is missing the new `ApplicationEventPublisher` argument.

claims-service - after fixing ONLY the planted compile error

```
$ mvn test
[ERROR] Failures / Errors:
[ERROR]   KafkaDomainEventPublisherTest.should_PropagateValidationException_When_ValidatorRejectsInvalidClaimStatus:214
[ERROR]   KafkaDomainEventPublisherTest.should_PropagateValidationException_When_ValidatorRejectsMissingRequiredField:191
[ERROR]   KafkaDomainEventPublisherTest.should_ThrowIllegalStateException_When_SerializationFails:141
[ERROR]   KafkaDomainEventPublisherTest.should_CallValidatorWithCorrectArgs_When_ClaimSubmittedEventPublished:167 » NullPointer Cannot
invoke "com.fas ...
[ERROR]   KafkaDomainEventPublisherTest.should_PublishEventToKafka_When_ClaimStatusChangedEventProvided:90 » NullPointer Cannot invoke
"com.fasterxml. ...
[ERROR]   KafkaDomainEventPublisherTest.should_PublishEventToKafka_When_ClaimSubmittedEventProvided:67 » NullPointer Cannot invoke
"com.fasterxml.jack ...
[ERROR]   CreateClaimUseCaseTest.should_CreateClaim_When_ValidCommandProvided:66 » NullPointer
[ERROR]   CreateClaimUseCaseTest.should_PublishClaimSubmittedEvent_When_ClaimCreated:109 » NullPointer
[ERROR]   CreateClaimUseCaseTest.should_SetStatusToSubmitted_When_ClaimCreated:129 » NullPointer
[ERROR] Tests run: 238, Failures: 3, Errors: 6, Skipped: 0
[INFO] BUILD FAILURE
```

Figure 11.2: Once it compiled, two more stale classes surfaced with 9 failures.

code/claims-service/src/test/java/com/example/demo/adapter/out/messaging/KafkaDomainEventPublisherTest.java (as received, commit 1dea234)

```
62          when(objectMapper.writeValueAsString(event)).thenReturn("{\"eventId\":\"test\"}");
63          doNothing().when(schemaValidator).validate(anyString(), anyString());
64          when(kafkaTemplate.send(anyString(), anyString(), anyString()))
65                  .thenReturn(CompletableFuture.completedFuture(null));
66
67          publisher.publish(event);
68
69          verify(kafkaTemplate).send(eq("claim-events"), eq(claimId.toString()), eq("{\"eventId\":\"test\"}"));
70          verify(schemaValidator).validate("{\"eventId\":\"test\"}", "ClaimSubmittedMessage");
```

Shipped test mocks writeValueAsString(event) - production now builds the envelope with createObjectNode(), so the mock returns null.

Figure 11.3: Root cause in the publisher test: it mocks a method production no longer calls.

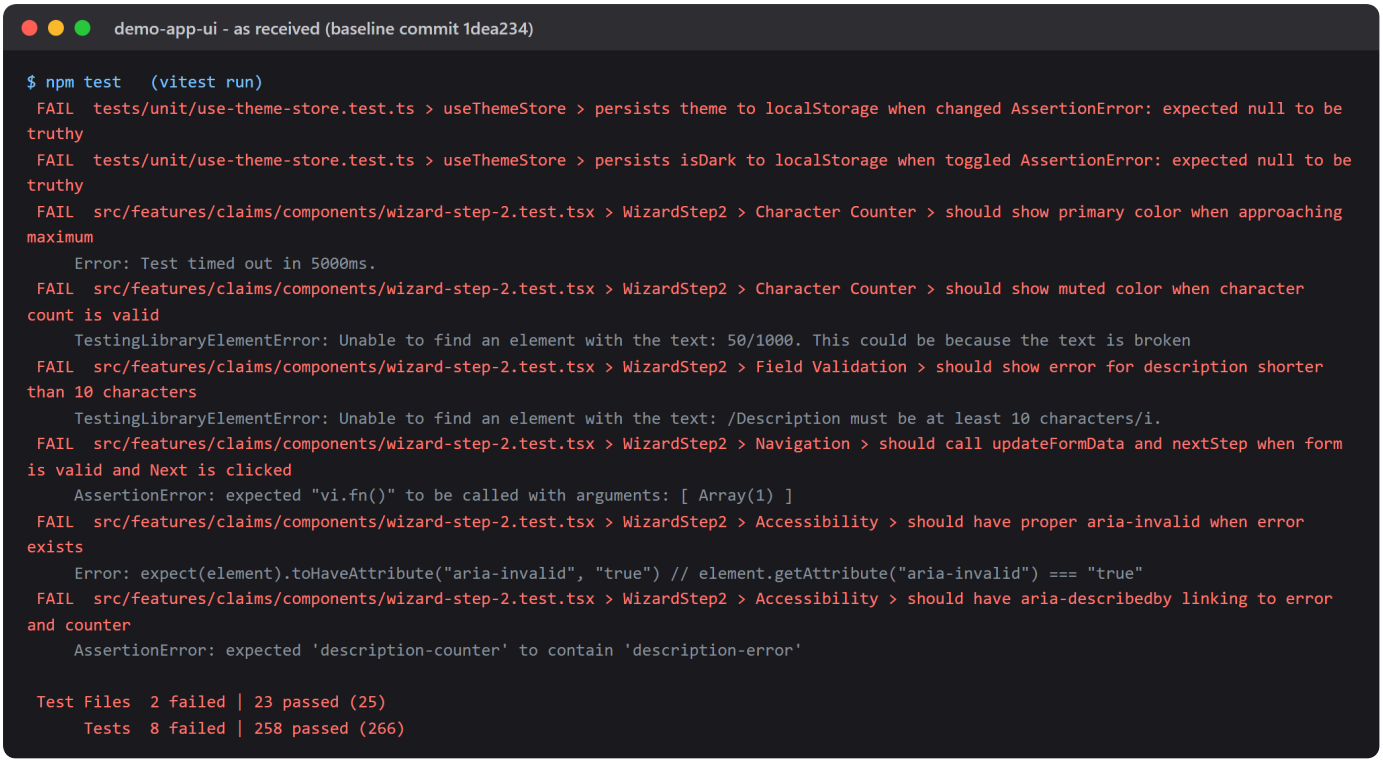


Figure 11.4: Baseline UI run: 2 theme-store failures and 6 wizard failures.

Root cause of the wizard failures. One test typed 950 characters one keystroke at a time, exceeded the 5-second timeout, and leaked keystrokes into five later tests. I changed it to paste the text, which is still a real user action, instead of raising the timeout.

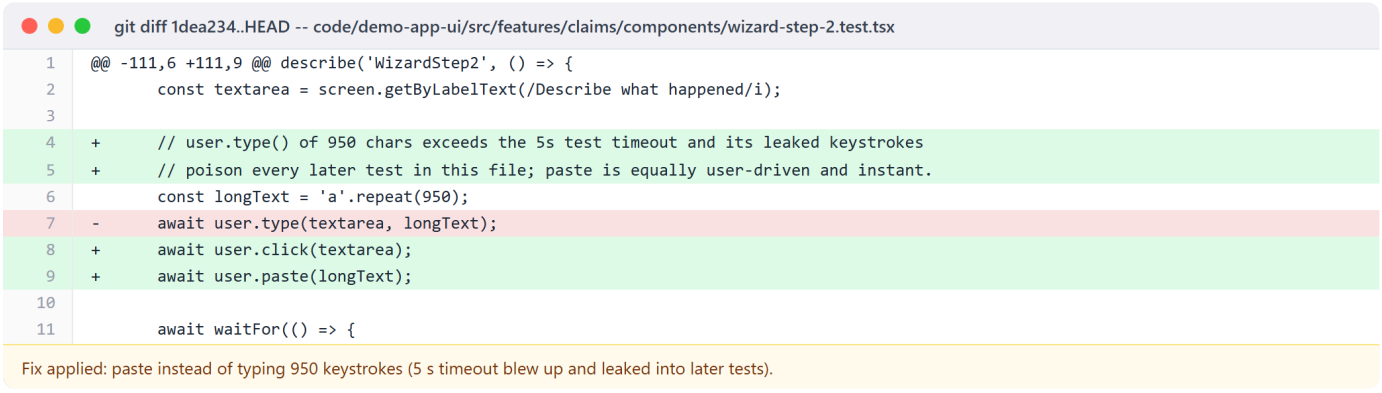


Figure 11.5: Fix applied.

Fixes applied for the claims service. I supplied the missing `ApplicationEventPublisher` to `UpdateClaimStatusUseCaseTest`, which was the planted error. `CreateClaimUseCaseTest` got the same missing mock, and it now asserts that the event is published and then cleared. I rewrote `KafkaDomainEventPublisherTest` with a real JSON mapper, so it asserts the actual Kafka envelope the BFF depends on.

12. Finding 10: Shared bearer token across concurrent requests

Medium **Open**

The BFF keeps one HTTP client per controller and overwrites its bearer token on every request. Under concurrency, a call to the claims service could carry another user's token. Ten parallel list-claims calls all succeeded on the live stack. That suggests the race window is narrow; it does not prove there is no race. The next step is a mixed-role concurrency test.

code/bff-service/src/main/java/com/example/demo/adapter/in/web/AdminController.java

```
148     private void setServiceAccountBearerToken() {
149         String token = serviceAccountTokenProvider.getServiceAccountToken();
150         apiClient.setBearerToken(token);
151     }
152
153     private void setUserBearerToken() {
154         var authentication = org.springframework.security.core.context.SecurityContextHolder
155                 .getContext().getAuthentication();
156         if (authentication != null && authentication.getPrincipal() instanceof org.springframework.security.oauth2.jwt.Jwt jwt) {
157             apiClient.setBearerToken(jwt.getTokenValue());
158         } else {
159             throw new IllegalStateException("No user JWT token found in security context");
```

One shared ApiClient per controller; each request **overwrites** its bearer token. Concurrent requests can send another user's token.

Figure 12.1: The token is set on a shared client instance.

Fix. Resolve the token per request, for example with an interceptor that reads the current security context.

13. Finding 11: `changedBy` records the claim owner instead of the admin

Medium **Open**

The schema says `changedBy` is the admin who changed the status, but the domain fills it with the claim owner. Owner notifications only work because the BFF uses this field as the notification target, so a naive fix would break them.

code/claims-service/src/main/java/com/example/demo/domain/claim/Claim.java

```
130         this.domainEvents.add(new ClaimStatusChanged(
131             UUID.randomUUID().toString(),
132             this.claimId,
133             oldStatus,
134             newStatus,
135             this.userId,
136             Instant.now()
137         ));
```

changedBy is filled with the claim **owner** (this.userId), not the admin who made the change.

Figure 13.1: `this.userId` is the claim owner.

Fix. Carry both `ownerId` and `changedBy` in the event, and route notifications on `ownerId`.

14. Finding 12: Theme preference lost in every new tab

Low Fixed

git diff 1dea234..HEAD -- code/demo-app-ui/src/features/theme/stores/use-theme-store.ts

```
@@ -23,5 +23,7 @@ export const useThemeStore = create<ThemeStore>()(
     {
       name: 'theme-storage',
-      storage: createJSONStorage(() => sessionStorage),
+      // Shipped tests specify localStorage (theme should survive new tabs/sessions);
+      // implementation used sessionStorage, silently resetting user preference.
+      storage: createJSONStorage(() => localStorage),
     }
   )
```

Fix applied: theme preference now persists in localStorage, as the tests specify.

Figure 14.1: One-line application fix to match the shipped tests' intent.

15. Finding 13: E2E runner broken because the setup file is missing

Low Fixed

code/demo-app-ui/playwright.config.ts (as received, commit 1dea234)

```
    baseURL: 'http://localhost:3001',
    trace: 'on-first-retry',
  },
  globalSetup: './tests/e2e/global-setup.ts',
  projects: [
```

Shipped config points to a global-setup file that did not exist in the repo.

Figure 15.1: The config referenced a file that did not exist. I added it. It logs in each seeded user once and warms every route, so `next dev` cold compiles cannot time out.

16. Finding 14: Unpinned Testcontainers images break the integration tests

Low Fixed

`postgres:latest` now resolves to Postgres 18.6, which rejects the JVM's default zone name `Asia/Calcutta` (the default on an Indian Windows machine). Two full-context tests errored (Figure 2.3). The test I left untouched already pinned `postgres:15-alpine`, the compose version.

git diff 23e08d9..HEAD -- code/claims-service/src/test/java/com/example/demo/TestcontainersConfiguration.java

```
@@ -24,5 +24,5 @@ public class TestcontainersConfiguration {
     @ServiceConnection
     PostgreSQLContainer postgresContainer() {
-        return new PostgreSQLContainer(DockerImageName.parse("postgres:latest"))
+        return new PostgreSQLContainer(DockerImageName.parse("postgres:15-alpine") /* match docker-compose.infra.yml; :latest (PG 1
                 .withReuse(true);
     }
@@ -31,5 +31,5 @@ public class TestcontainersConfiguration {
     @ServiceConnection(name = "redis")
     GenericContainer<?> redisContainer() {
-        return new GenericContainer<>(DockerImageName.parse("redis:latest"))
+        return new GenericContainer<>(DockerImageName.parse("redis:7-alpine") /* match docker-compose.infra.yml */)
                 .withExposedPorts(6379)
                 .withCommand("redis-server", "--databases", "32")
```

Fix applied: pin to the versions docker-compose.infra.yml runs.

Figure 16.1: Fix applied in both services. Integration now passes 12 of 12.

17. Finding 15: Concurrent logins for the same account are rejected

Low Open

Ten parallel logins with correct credentials mostly fail with 401 "Invalid email or password", whether they go through the BFF or straight to Keycloak. Ten distinct users in parallel all succeed. The realm has brute-force protection enabled. Real users hit this with a double submit or simultaneous logins from two devices.

Parallel logins: one shared account vs distinct accounts

```
$ node verify-concurrency.cjs 10   +   direct Keycloak token grants
sequential logins x10: {"200":10}
parallel   logins x10: {"200":1,"401":9}
sequential GET /api/claims x10: {"200":10}
parallel   GET /api/claims x10: {"200":10}
# 10 parallel password grants for the SAME account, straight to Keycloak
      3 200
      7 401
# 10 parallel grants for 10 DISTINCT accounts (seeded with load/seed-load-users.sh), straight to Keycloak
     10 200
# 10 parallel logins for 10 DISTINCT accounts, through the BFF
     10 200
```

Figure 17.1: Same account versus distinct accounts.

> **Attribution corrected.** I first suspected finding 10, the shared token. Calling Keycloak directly ruled that out, so I did not count this as evidence for finding 10.

18. Findings 16–18: smaller issues

Low Open

- **16: `start-cdc.sh` reports failure on a clean, successful run.** The CDC compose file's init container registers the connector, then the script registers it again and exits 1 on the 409. The connector is actually RUNNING.
- **17: The status-change error detail is discarded.** The BFF returns an empty 400, so "Cannot transition from X to Y" never reaches the admin (Figure 18.1).
- **18: Smaller smells.**
  - Money is declared as `double` in both APIs.
  - The domain accepts 0.001, while the API floor is 0.01.
  - The WebSocket accepts the JWT in the URL.
  - The access token lives 24 h, its cookie 5 min.
  - The UI truncates at 50 characters, the API at 100.
  - The test Keycloak is 26.1.4, compose runs 26.5.2.

code/bff-service/src/main/java/com/example/demo/adapter/in/web/AdminController.java

```
133             return ResponseEntity.ok(adminClaimResponseMapper.toClaimResponse(claimsResponse));
134         } catch (ApiException exception) {
135             if (exception.getCode() == 400) {
136                 return ResponseEntity.badRequest().build();
137             }
```

The claims-service ProblemDetail ("Cannot transition from X to Y") is discarded and an empty 400 is returned.

Figure 18.1: An empty 400 is returned for an invalid status transition.

19. The application under test

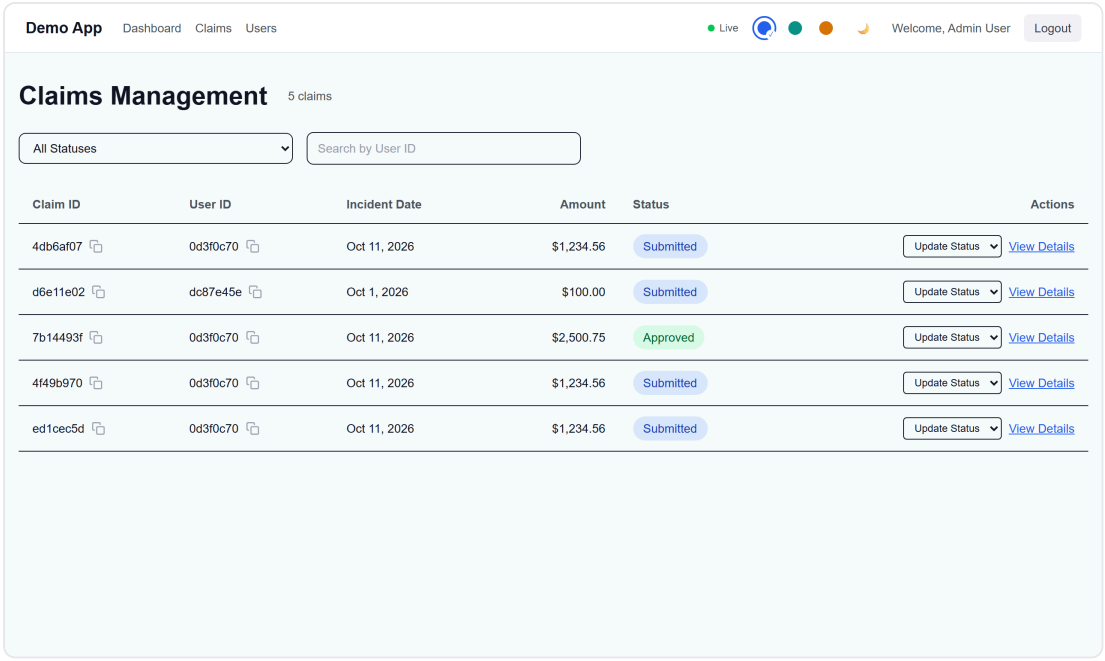


Figure 19.1: Admin claims management on the live stack, showing claims created by the E2E and live probes. "Live" means the WebSocket is connected.

New component tests - login page and claims table

```
$ npx vitest run login-page.test.tsx claims-table.test.tsx --reporter=verbose
✓ ClaimsTable > renders one row per claim with formatted amount, date, and status label 204ms
✓ ClaimsTable > truncates long descriptions but keeps the full text available as a tooltip 5ms
✓ ClaimsTable > highlights only the row whose claim was just updated in real time 6ms
✓ ClaimsTable > invokes onViewDetails with the full claim id 79ms
✓ ClaimsTable > copies the full claim id to the clipboard from the copy affordance 94ms
✓ LoginPage > submits credentials to the BFF login endpoint with cookies enabled 735ms
✓ LoginPage > stores the authenticated user in the auth store on success 632ms
✓ LoginPage > shows "Invalid email or password" on a 401 without storing a user 683ms
✓ LoginPage > shows a network error message when the BFF is unreachable 660ms
✓ LoginPage > blocks submission and shows a validation message for a malformed email 703ms
✓ LoginPage > requires a password before submitting 306ms
✓ LoginPage > announces the session-expired message when redirected with session_expired=true 17ms
Test Files  2 passed (2)
Tests  12 passed (12)
```

Figure 19.2: New component tests for the login page and claims table.

git diff 1dea234..HEAD -- code/bff-service/pom.xml

```
@@ -45,5 +45,5 @@
                <skipAsyncApiGeneration>true</skipAsyncApiGeneration>
                <pact.version>4.6.3</pact.version>
-               <surefire.excludedGroups>integration</surefire.excludedGroups>
+               <surefire.excludedGroups>integration,defect</surefire.excludedGroups>
        </properties>
        <dependencies>
```

Known-defect tests are tagged "defect" and kept out of the default run, so CI stays green and the bugs stay reproducible.

Figure 19.3: Defect tests are kept out of the default run, so CI stays green while each bug stays reproducible with one command.

20. What I would do next, in priority order

1. **Fix findings 1–5.** Four are configuration or one-line changes, and findings 1 and 2 are already validated live. The failing defect tests then turn green and stay as regression guards.
2. **Make setup reproducible.** Generate the API client in the build, pin every image, and add a CI job that builds and runs E2E from a fresh clone.
3. **Fix the CDC schema to match real Debezium output.** Then validate a captured event in CI with the existing validator.
4. **Pass claims-service errors through the BFF unchanged (findings 7 and 17).** Treat date-only fields as dates (finding 8).
5. **Write a mixed-role concurrency test for finding 10,** then move to per-request tokens.
6. **Run k6 at higher load against a production build** (`next start`), then set real thresholds.

Commit history

```
$ git log --oneline --reverse
1dea234 chore: baseline import of provided claims application (as received)
fffee08 chore: ignore build outputs and macOS metadata
833c777 docs: add application risk assessment and test strategy
b97ea89 test(claims): repair stale suite so the baseline actually runs
9d0e427 test(claims): exact boundary values and full status transition matrix
1c5d79a test(bff): content-level Kafka consumer tests and logout CSRF contract
40d94cd test(ui): login page and claims table component tests; fix failing shipped tests
f3a6314 test(e2e): Playwright journeys with page objects and the missing global setup
a93ed2a test(load): k6 smoke script for list-claims and cached dashboard
fe44881 docs: AI working journal, verified results and run guide
23e08d9 docs: add QA findings report (PDF) with evidence screenshots
374945d test(e2e): make the Playwright suite pass against the live stack
f9fd6a0 test(integration): pin Postgres and Redis Testcontainers images to the compose versions
f61e72b test(ui): pin the incident-date off-by-one for users west of UTC
573471f test(load): make the k6 run meaningful and green on the live stack
09f541e docs: record live-stack verification and seven new findings
e86fcf3 chore: keep LF line endings for shell scripts
```

Figure 20.1: Commit history, from the as-received baseline onward.